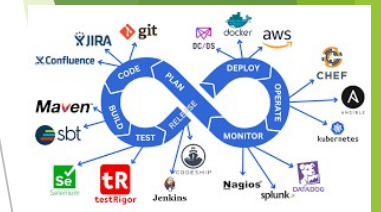


Setting up the CI/CD

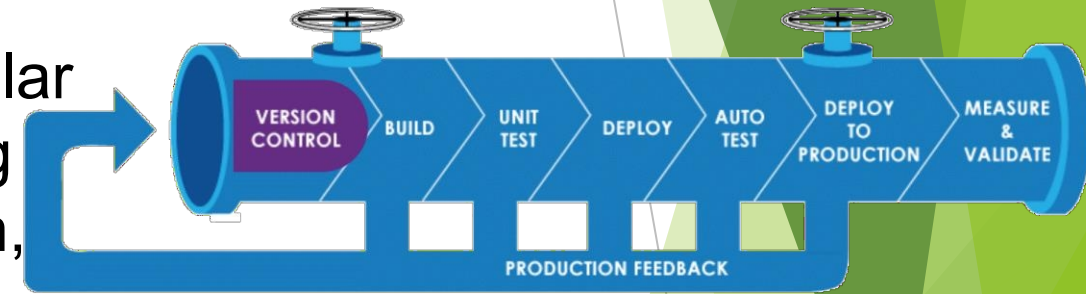
Continuous integration & deployment



What is CI/CD?

In a world where technology is evolving at a breakneck pace, software development teams need to adapt and innovate quickly to remain competitive. It is in this context that the concept of **CI/CD** - Continuous Integration and Continuous Deployment - has emerged as a fundamental pillar of modern development practices. By integrating automated processes that promote collaboration, quality and speed, CI/CD is transforming the way software is designed, tested and deployed.

CI/CD, or **Continuous Integration/Continuous Deployment**, is a method used in software development to automate and improve the development process. code delivery.



Continuous integration / L'intégration continues.

It highlights two key concepts:

- ▶ Allows developers to merge their code changes into a main branch on a frequent basis (at least once a day).
- ▶ Every time a developer pushes code, a series of automated tests are run to check that the code is functional and doesn't break existing functionality.

Continuous Deployment (CD)

- ▶ Extends continuous integration by automating the deployment of applications with each validated change in the main branch. Once the code passes all the tests, it is automatically deployed in a production or staging environment.
- ▶ Reduced time from development to production.
- ▶ Assurance that the code is always ready for deployment.

Continuous Deployment (CD)

- ▶ Extends continuous integration by automating the deployment of applications with each validated change in the main branch. Once the code passes all the tests, it is automatically deployed in a production or staging environment.

Automated Test Execution

- ▶ The series of automated tests as part of continuous integration (CI) is usually run on **the server** side. Here are some details:
- ▶ **CI server :**
 - ▶ When a developer pushes code to a repository (for example, on GitHub or GitLab), a **continuous integration server** (such as Jenkins, CircleCI, or GitLab CI) automatically triggers a series of tests.
 - ▶ These tests can include unit tests, integration tests and tests functional.
 - ▶ **Test environment :**
 - ▶ Tests are often run in an isolated environment that reflects the production environment to ensure that the results are representative.
- ▶ **Notifications and Reports:**
 - ▶ Once the tests have been run, the CI server generates reports that inform developers of the success or failure of the tests. In the event of failure, developers receive notifications so that they can correct problems quickly.

Why use CI/CD?

- ▶ **Why use CI/CD?**
- ▶ **Quality:** Automated testing ensures that new changes do not introduce bugs.
- ▶ **Speed:** Development cycles are accelerated, enabling frequent updates.
- ▶ **Collaboration:** Fosters a culture of teamwork and transparency.

Common tools

- ▶ **CI:** Jenkins, GitLab CI, Travis CI, CircleCI, GitHub action.
- ▶ **CD:** Kubernetes, Docker, AWS CodeDeploy, Azure DevOps.

The cicd with Gitlab

- ▶ Like other similar platforms such as GitHub, GitLab is a **Git repository manager** that facilitates collaboration within development teams, based on the Ruby language.
- ▶ GitLab also offers integrated features for **continuous integration and continuous delivery** (CI/CD), grouped together under the name **GitLab CI/CD**.
- ▶ Each time a code *push* is performed, GitLab can run a pipeline of scripts to test the changes before validating and integrating them into the project.
- ▶ GitLab's Continuous Delivery (CD) also gives us the ability to automatically deploy our applications to production with every push of code.

The cicd with Gitlab

The pipeline

- ▶ This is a sequence of processes that run automatically when changes are made to the source code.
- ▶ A pipeline is triggered by specific events, such as a commit on a branch, a merge request or a schedule.

The cicd with Gitlab

Configuring the CI/CD pipeline

- ▶ In GitLab, a pipeline is defined by a configuration file (generally **.gitlab-ci.yml**), which specifies the steps to be executed. To be continued.
- ▶ This file is located at the root of your Git repository.
- ▶ The CI/CD pipeline for this file is triggered automatically each time the code is pushed.

```
stages:
  - build
  - test
  - deploy

build:
  stage: build
  script:
    - echo "Building the application..."
    - ./build.sh

test:
  stage: test
  script:
    - echo "Running tests..."
    - ./run_tests.sh

deploy:
  stage: deploy
  script:
    - echo "Deploying the application..."
    - ./deploy.sh
  only:
    - main # Déployer uniquement depuis la branche principale
```

The cicd with Gitlab

Pipeline triggers

These are mechanisms that initiate the execution of a pipeline. They automate the continuous integration and continuous deployment (CI/CD) processes by responding to various events.

- ▶ **Pushing Code (Push Events):** A pipeline is triggered when a commit is pushed into a branch of the depot.
- ▶ **Merge Requests:** A pipeline can also be triggered when a merge request is created or updated.
- ▶ **scheduling (Scheduled Pipelines):** You can configure pipelines to run at specific times on a recurring basis (for example, every day at a certain time).

The cird with Gitlab

Pipeline triggers

- ▶ **Webhooks:** GitLab can receive notifications via webhooks. can trigger pipelines in response to external events.
- ▶ **API:** GitLab exposes a REST API that allows pipelines to be triggered manually via HTTP requests.
- ▶ **Manual Triggers:** Pipelines can also be manually triggered via the GitLab user interface.
- ▶ **Tag Changes (Tag Events):** A pipeline can be triggered when a tag is created or updated in the repository.
- ▶ **Conditions and Rules:** Rules in the `.gitlab-ci.yml` file can specify conditions to trigger jobs or pipelines based on specific events (for example, changes to certain branches or files).

The cicd with Gitlab

The runners

- ▶ A runner is a program that runs on a machine and waits for instructions from GitLab to execute specific tasks (jobs). Each time a pipeline is triggered, GitLab sends the jobs to a runner, which executes them.

The cicd with Gitlab

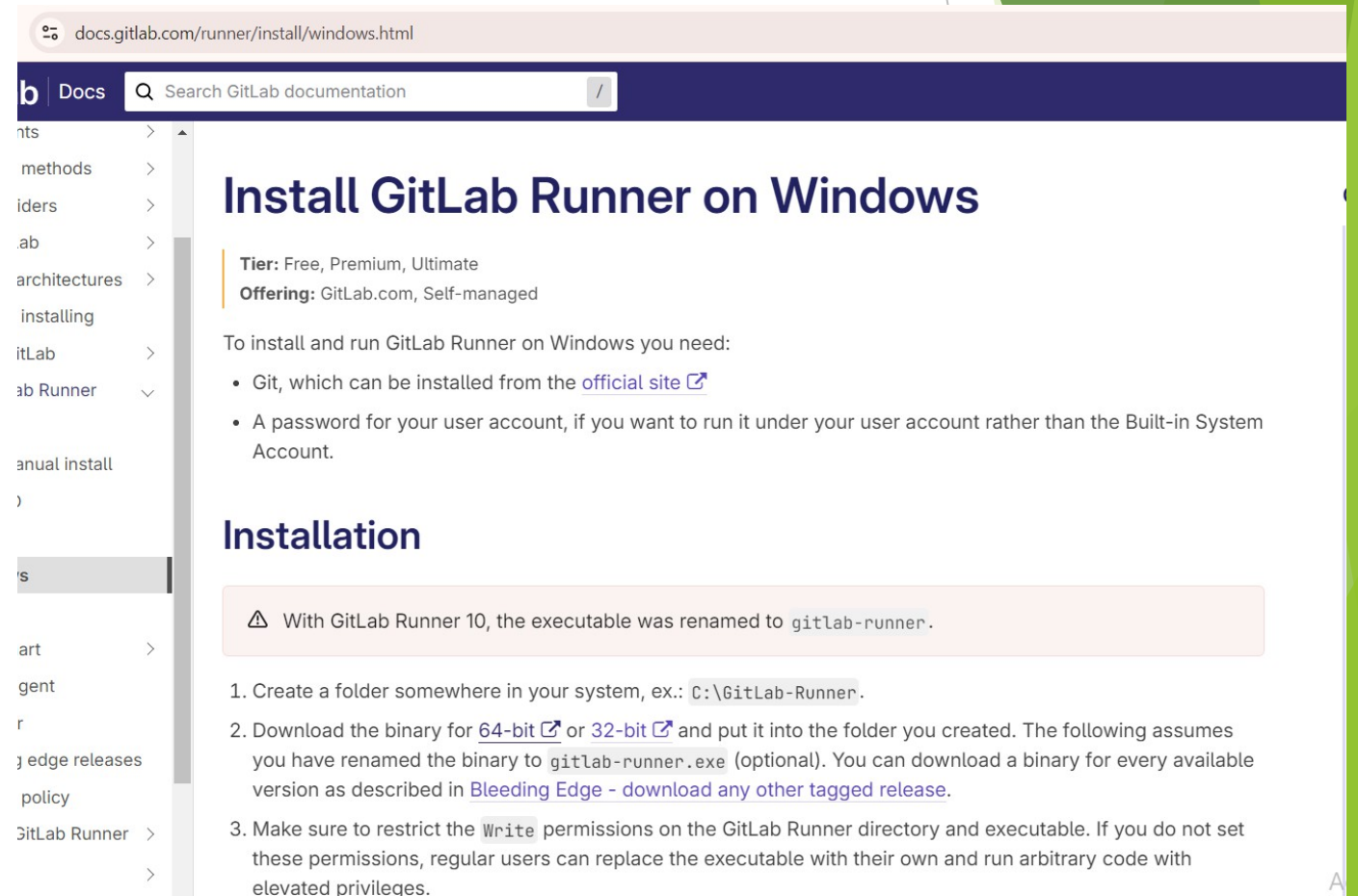
Types of Runners

- ▶ There are several types of runners in GitLab:
- ▶ **Shared Runners:**
 - ▶ Runners that are available for all projects in a GitLab instance.
 - ▶ Ideal for projects that do not require specific configurations.
- ▶ **Specific Runners:**
 - ▶ Runners that are registered for a specific project or group.
 - ▶ Enables custom configurations and environments for particular needs.

The cicd with Gitlab

Installation of gitlab-runner.

- First download and install the gitlab-runner program on the host machine.
<https://docs.gitlab.com/runner/install/windows.html>
- To use a runner, you must first register it with your GitLab instance. Here's an overview of the steps: (practical)



The screenshot shows the GitLab documentation page for installing GitLab Runner on Windows. The browser address bar shows the URL `docs.gitlab.com/runner/install/windows.html`. The page has a dark blue header with the GitLab logo and a search bar. A sidebar on the left contains a navigation menu with categories like 'Docs', 'Getting started', 'Configuration', and 'Troubleshooting'. The main content area is titled 'Install GitLab Runner on Windows' and includes information about the runner's tiers and offerings. It lists the requirements for installation: Git and a user account password. The 'Installation' section provides a list of three steps: creating a folder, downloading the binary, and restricting permissions. A warning box notes that the executable was renamed to `gitlab-runner` in version 10.

docs.gitlab.com/runner/install/windows.html

b Docs Q Search GitLab documentation /

Install GitLab Runner on Windows

Tier: Free, Premium, Ultimate
Offering: GitLab.com, Self-managed

To install and run GitLab Runner on Windows you need:

- Git, which can be installed from the [official site](#)
- A password for your user account, if you want to run it under your user account rather than the Built-in System Account.

Installation

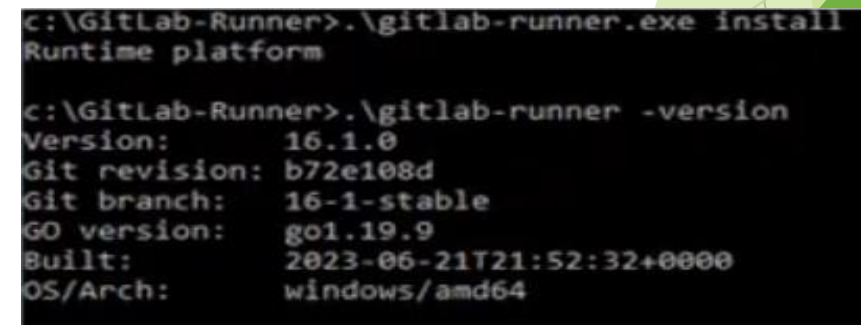
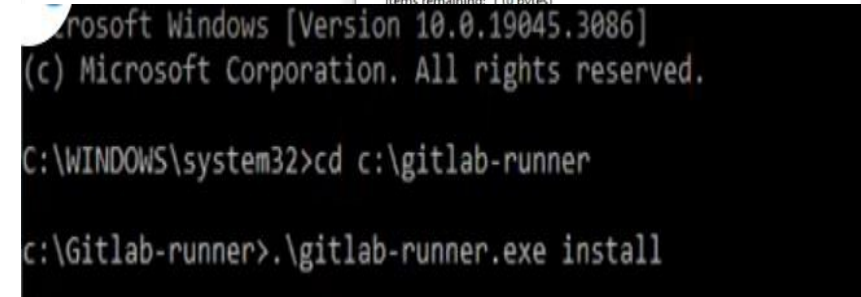
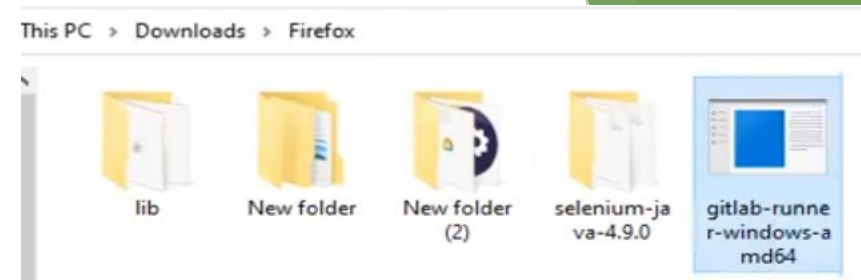
⚠ With GitLab Runner 10, the executable was renamed to `gitlab-runner`.

1. Create a folder somewhere in your system, ex.: `C:\GitLab-Runner`.
2. Download the binary for [64-bit](#) or [32-bit](#) and put it into the folder you created. The following assumes you have renamed the binary to `gitlab-runner.exe` (optional). You can download a binary for every available version as described in [Bleeding Edge - download any other tagged release](#).
3. Make sure to restrict the `Write` permissions on the GitLab Runner directory and executable. If you do not set these permissions, regular users can replace the executable with their own and run arbitrary code with elevated privileges.

The cicd with Gitlab

Installation of gitlab-runner.

- ▶ Create a folder called gitlab-runner in a location of your choice on your hard drive and paste your recently downloaded gitlab-runner into it.
- ▶ Rename the file to: gitlab- runner.exe
- ▶ Open the command prompt on the folder and run the command **.\gitlab-runner.exe install** as shown in the last figure.
- ▶ Then run the command
- ▶ **gitlab-runner.exe -v**

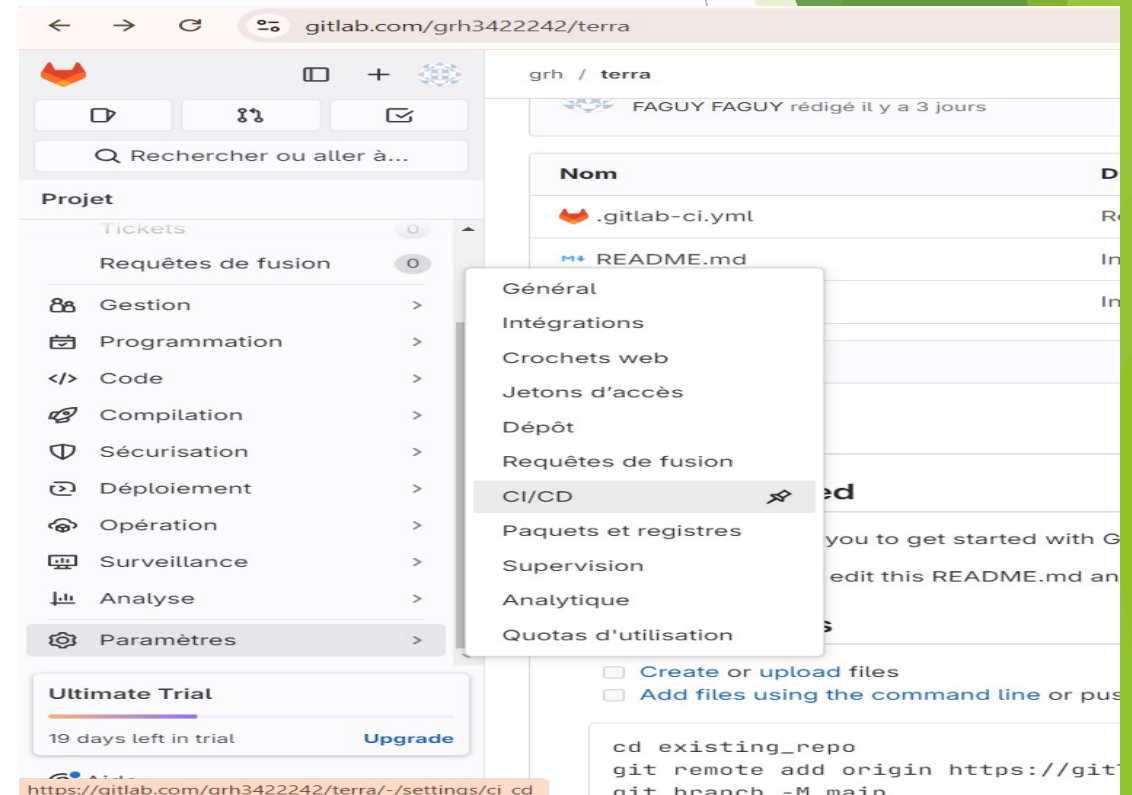


The cicd with Gitlab

Registration stage for a specific runner.

- ▶ First you type the command **gitlab-runner.exe register** as shown in the first image and follow the command line registration wizard.
- ▶ During registration you will be asked to enter a registration token. This is a unique code used to register a runner with your GitLab instance. To obtain this code, go to your project settings in gitlab, then to CI/CD and pull down the runners section

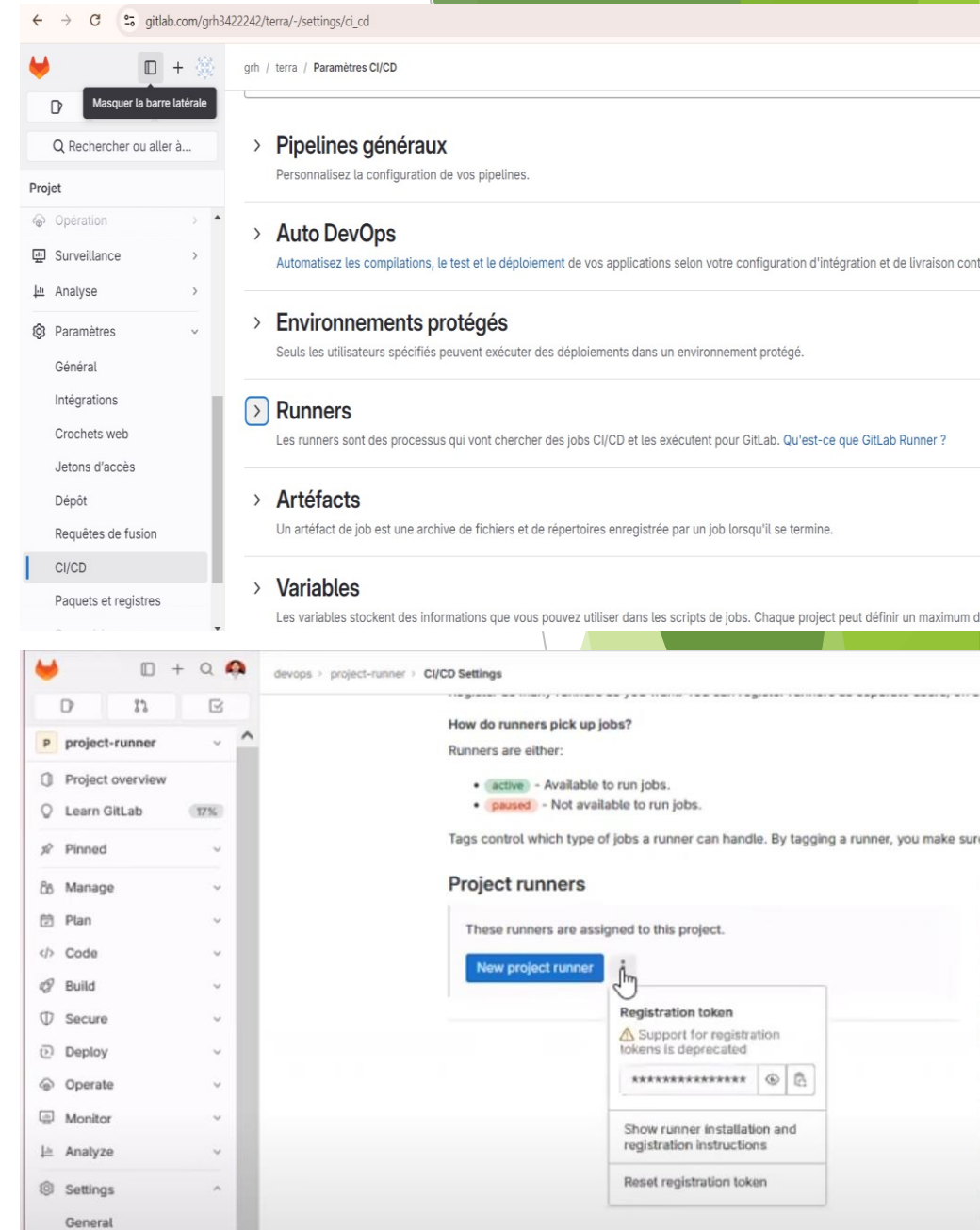
```
WINDOWS\system32>cd c:\gitlab-runner
c:\Gitlab-runner>.\gitlab-runner.exe install
Runtime platform                                arch=amd64 os=windows pid=9272
c:\Gitlab-runner>.\gitlab-runner -v
Version:      16.1.0
Git revision: b72e108d
Git branch:   16-1-stable
GO version:   go1.19.9
Built:        2023-06-21T21:52:32+0000
OS/Arch:      windows/amd64
c:\Gitlab-runner>.\gitlab-runner.exe register
Runtime platform                                arch=amd64 os=windows pid=1244
Created missing unique system ID                system_id=s_22b37f78dc8f
Enter the GitLab instance URL (for example, https://gitlab.com/):
https://gitlab.com/
```



The cicd with Gitlab

Step for registering a specific runner (continued).

- ▶ The image opposite is the CI/CD settings page for your gitlab project.
- ▶ Scroll down the runner section and, as shown in the last figure opposite, click on the three vertical buttons next to new project runner to obtain the registration token that is hidden by default.
- ▶ Copy the token and return to the command prompt to continue recording your first runner.



The cicd with Gitlab

Executors

When registering a command prompt runner, you are asked to choose an execution environment (**or executor**) for your pipeline.

Runners can use different types of executor to run jobs:

- ▶ **Shell:** Executes scripts directly in the shell.
- ▶ **Docker:** Runs jobs in Docker containers, enabling environments to be isolated.
- ▶ **Docker Machine:** Creates and manages Docker containers dynamically.
- ▶ **Kubernetes:** Runs jobs in a Kubernetes cluster.
- ▶ **Parallels, VirtualBox, etc. :** Other options for running on machines virtual.

The cicd with Gitlab

Runner configuration

Once your runner has been registered, a `config.toml` configuration file is generated in the `gitlab-runner` directory. This file contains the runners registered in your gitlab projects or project groups.

The cicd with Gitlab

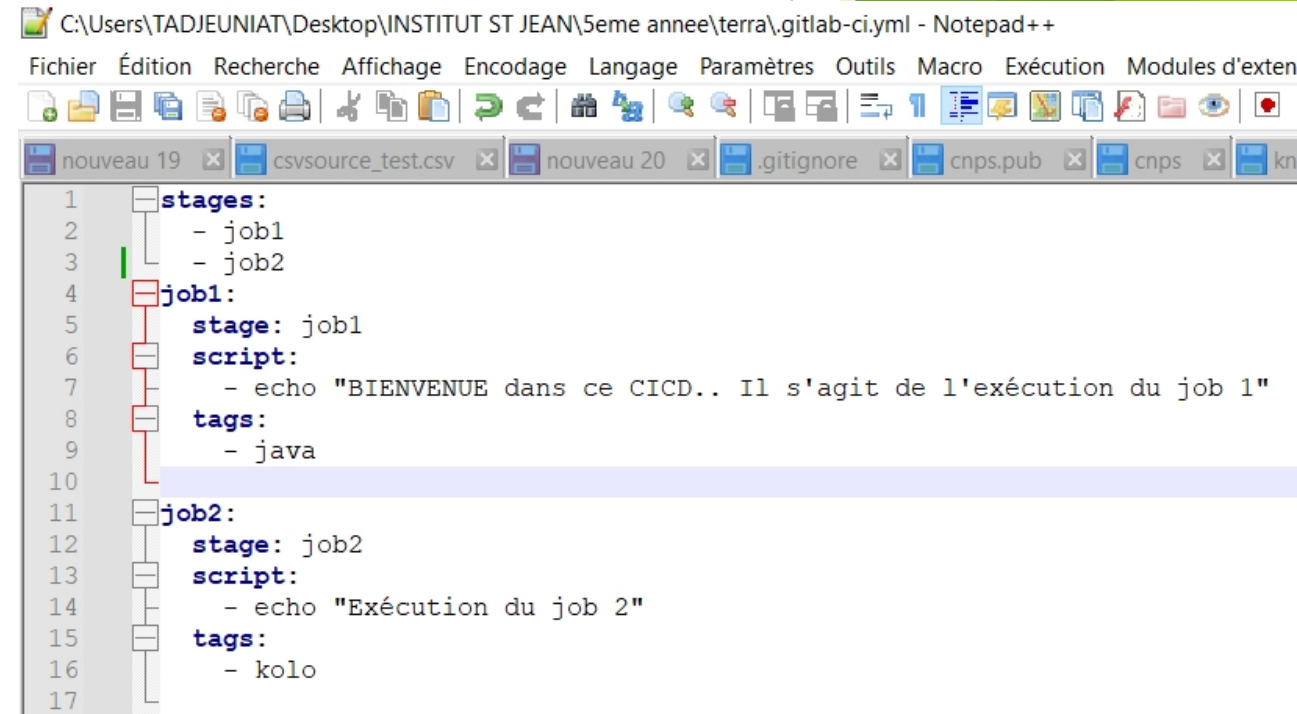
Advantages of Runners

- ▶ Scalability: You can add as many runners as you need to manage the workload.
- ▶ Flexibility: Runners can be configured as required specific to a project.
- ▶ Isolation: By using containers or virtual machines, jobs can run in isolated environments, reducing conflicts.

The cicd with Gitlab

Structuring the Gitlab-ci.yml file

- ▶ Basically, the Gitlab-ci.yml file defines a set of jobs that will be executed in our CICD pipeline.
- ▶ A GitLab CI job is a unit of work defined in the .gitlab-ci.yml configuration file. Each job represents a task to be carried out as part of a CI/CD (Continuous Integration/Continuous Deployment) pipeline. Jobs can include steps such as compilation, testing, deployment and other operations required to build and deliver an application.

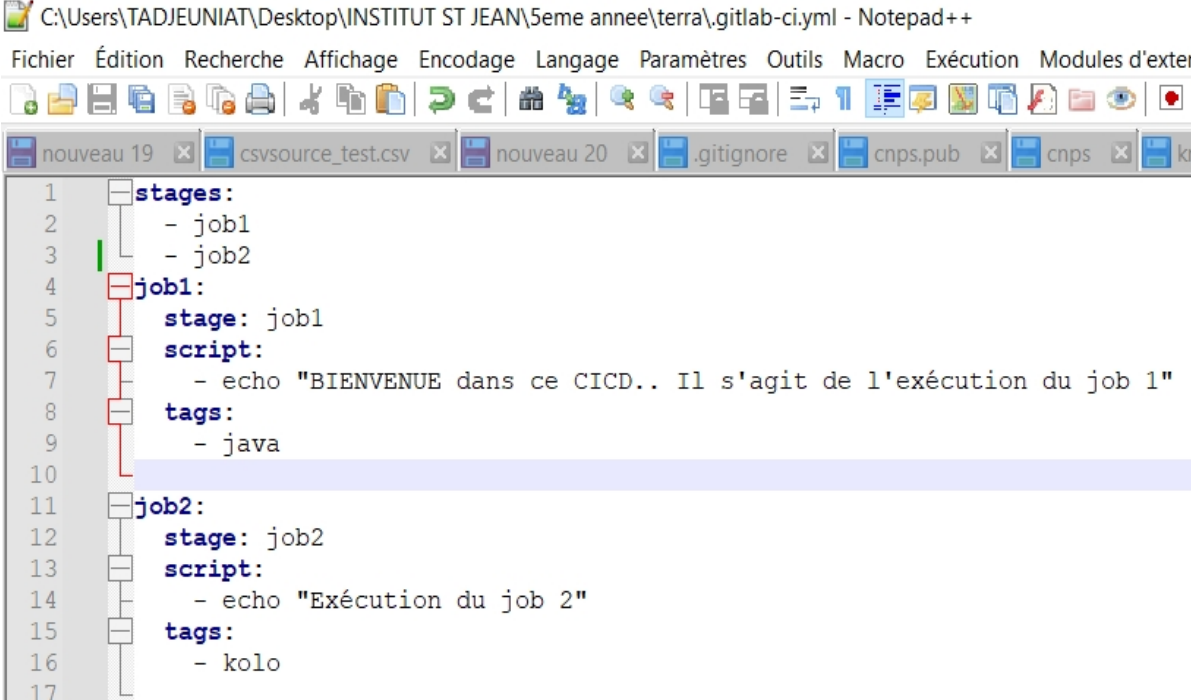


```
C:\Users\TADJEUNIAT\Desktop\INSTITUT ST JEAN\5eme annee\terra\gitlab-ci.yml - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'exten
nouveau 19 x  csvsource_test.csv x  nouveau 20 x  .gitignore x  cnps.pub x  cnps x  kn
1  stages:
2    - job1
3    - job2
4  job1:
5    stage: job1
6    script:
7      - echo "BIENVENUE dans ce CICD.. Il s'agit de l'exécution du job 1"
8    tags:
9      - java
10
11 job2:
12   stage: job2
13   script:
14     - echo "Exécution du job 2"
15   tags:
16     - kolo
17
```


The cicd with Gitlab

Structuring the Gitlab-ci.yml file (continued)

- Jobs are often organised into **stages**. Each **stage** can contain several jobs that run in parallel or sequentially. For example, you might have stages for build, test, and deploy.
- A job is a specific task to be executed as part of an internship. Each job may contain scripts or commands to run, environment configurations, dependencies, and other parameters.

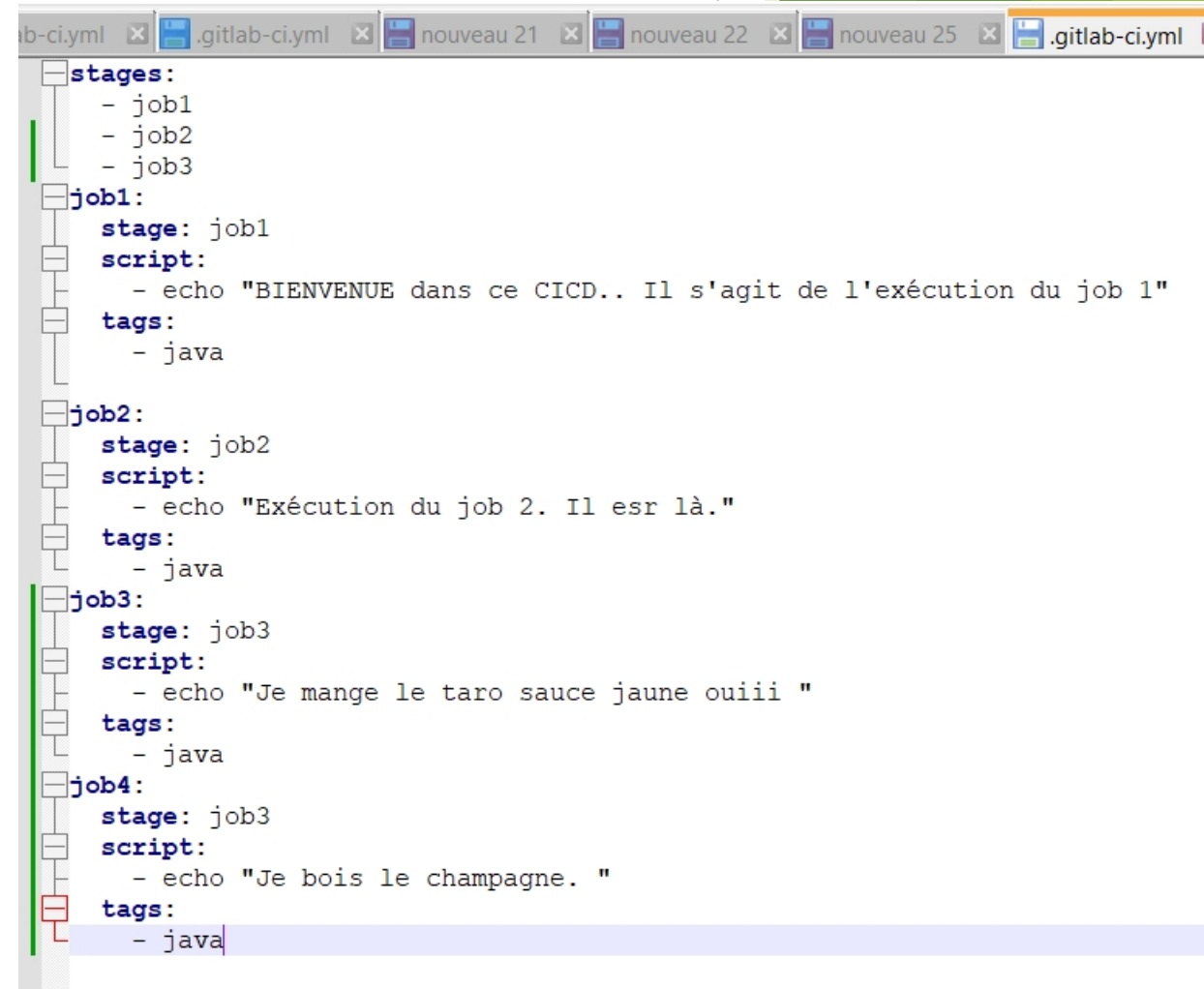


```
C:\Users\TADJEUNIAT\Desktop\INSTITUT ST JEAN\5eme annee\terra\.gitlab-ci.yml - Notepad++
Fichier  Édition  Recherche  Affichage  Encodage  Langage  Paramètres  Outils  Macro  Exécution  Modules d'exten
nouveau 19 x csvsource_test.csv x nouveau 20 x .gitignore x cnps.pub x cnps x kn
1  stages:
2    - job1
3    - job2
4  job1:
5    stage: job1
6    script:
7      - echo "BIENVENUE dans ce CICD.. Il s'agit de l'exécution du job 1"
8    tags:
9      - java
10
11 job2:
12   stage: job2
13   script:
14     - echo "Exécution du job 2"
15   tags:
16     - kolo
17
```


The cicd with Gitlab

Structuring the Gitlab-ci.yml file (continued)

- Jobs in the same stage can run in parallel. This speeds up the overall pipeline process, as several jobs can be run at the same time.

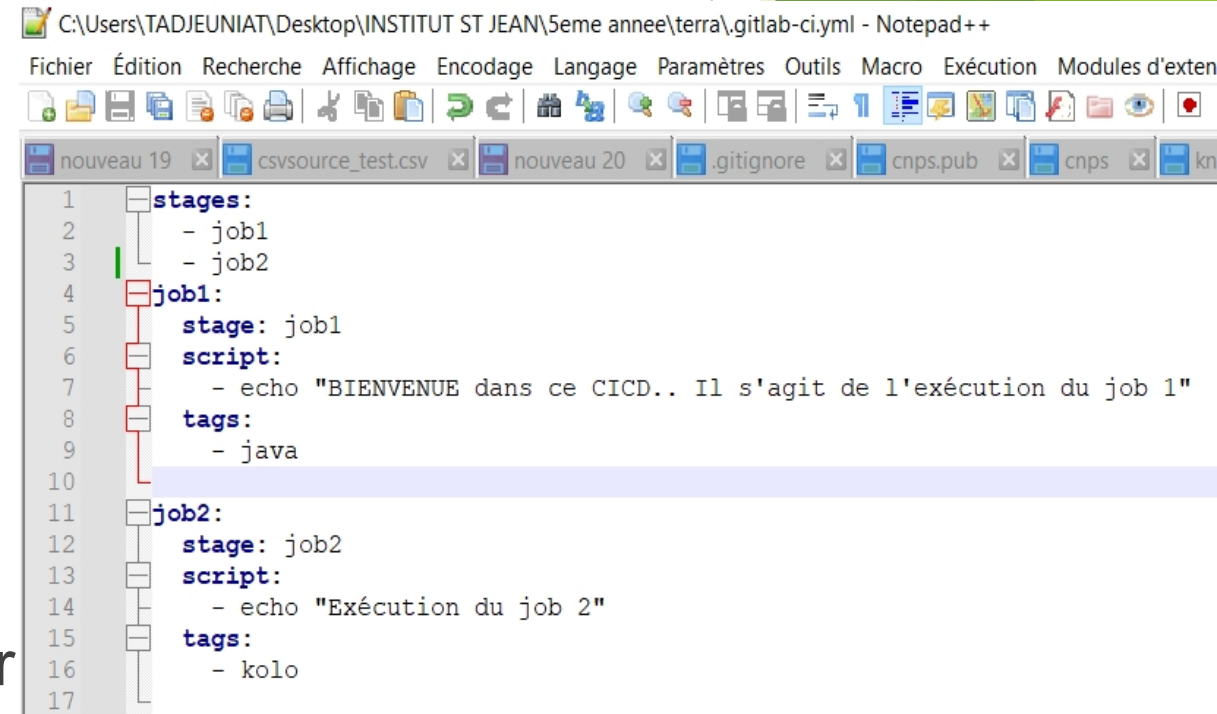


```
stages:
  - job1
  - job2
  - job3
job1:
  stage: job1
  script:
    - echo "BIENVENUE dans ce CICD.. Il s'agit de l'exécution du job 1"
  tags:
    - java
job2:
  stage: job2
  script:
    - echo "Exécution du job 2. Il esr là."
  tags:
    - java
job3:
  stage: job3
  script:
    - echo "Je mange le taro sauce jaune ouiii "
  tags:
    - java
job4:
  stage: job3
  script:
    - echo "Je bois le champagne. "
  tags:
    - java
```

The cicd with Gitlab

Structuring the Gitlab-ci.yml file (continued)

- ▶ In a job, you can define tags to indicate that it should be run by a specific runner that has those tags. This is particularly useful when you have runners configured for specific environments or configurations (for example, runners with particular software or libraries).
- ▶ Runners in GitLab can be configured with tags. These tags identify the capabilities or features of the runner.



```
1 stages:
2   - job1
3   - job2
4 job1:
5   stage: job1
6   script:
7     - echo "BIENVENUE dans ce CICD.. Il s'agit de l'exécution du job 1"
8   tags:
9     - java
10
11 job2:
12   stage: job2
13   script:
14     - echo "Exécution du job 2"
15   tags:
16     - kolo
17
```

The cicd with Gitlab

Structuring the Gitlab-ci.yml file

Other components

- ▶ However there are several other components of our pipeline apart from the jobs and internships mentioned above.
- ▶ These components are used according to the needs and as we change the environment in which we run our pipelines.

The cicd with Gitlab

Structuring the Gitlab-ci.yml file

Other components (continued)

- ▶ **image:** Specifies the Docker image to use for all jobs. This is particularly useful if all your jobs run in the same environment.
- ▶ **before_script:** Defines commands that will be executed before each job. This is useful for common stages such as outbuilding installations.
- ▶ **after_script:** Defines commands to be executed after each job, regardless of whether the job succeeded or failed.
- ▶ **variables:** Used to define default environment variables that will be available in all jobs.
- ▶ **retry:** Specifies the number of retries if a job fails.
- ▶ **artifacts:** Defines the files or directories to be retained after a job has run.

The cicd with Gitlab

Structuring the Gitlab-ci.yml file

Other components (Illustrative example)

- ▶ **Docker image:** all jobs will use the node:latest image, which ensures that the execution environment is the same for each job.
- ▶ **Before Script:** Before each job, npm install will be run, which ensures that all the necessary dependencies are installed without having to redefine it in each job.
- ▶ **Variables:** The NODE_ENV variable is defined as production and is accessible in all jobs.
- ▶ **Specific jobs:** build, test, and deploy jobs inherit the configurations defined in default, but you can still add specific configurations to each job if necessary.

```
1  default:
2    image: "node:latest"
3    before_script:
4      - npm install
5    variables:
6      NODE_ENV: "production"
7
8  stages:
9    - build
10   - test
11   - deploy
12
13  build:
14    stage: build
15    script:
16      - echo "Building the project..."
17      - npm run build
18
19  test:
20    stage: test
21    script:
22      - echo "Running tests..."
23      - npm test
24
25  deploy:
26    stage: deploy
27    script:
28      - echo "Deploying the project..."
29    after_script:
30      - echo "Deployment completed."
```

Practical work on setting up a simple CI/CD pipeline.

Our pipeline consists of a set of 3 steps for preparing taro in yellow sauce with the Pig's head:

- ▶ **Stage 1: Going to the market.**
 - ▶ Job1: Catch the neighbour's pig and confiscate its head
 - ▶ Job2: Bought taro, red oil and condiments.
- ▶ **Step 2: Cooking**
 - ▶ Job3: Cleaning the pig's head
 - ▶ Job4: Mashing taro
 - ▶ Job5: Making the sauce.
- ▶ **Step 3: Test**
 - ▶ Job6: Check that there is not too much salt
- ▶ **Stage 4: Serving at the table (deployment)**
 - ▶ Job7: Serving food
 - ▶ Job8: Serving drinks

Our source code, which will trigger the CI/CD pipeline after a push, is simply the application program used to set up the CI/CD pipeline.
instead of a good dish of taro in yellow sauce.

TP for setting up a Simple CI/CD pipeline.

- ▶ Create a **taro** folder in your system.
Add a recipe.txt file named recipe (this is our source code).
- ▶ Create the **.gitlab-ci.yml** file at the root of your project.
- ▶ Initialise a local repository in this folder.
- ▶ Create a blank taro remote repository. Index and commit changes

```
1 stages:
2   - aller_au_marche
3   - faire_la_cuisine
4   - tester
5   - servir
6
7 job1:
8   stage: aller_au_marche
9   script:
10    - echo "Attraper le cochon du voisin et confisquer sa tête. "
11   tag:
12    - taro
13
14 job2:
15   stage: aller_au_marche
16   script:
17    - echo "Acheter le taro, l'huile rouge et les condiments."
18   tag:
19    - taro
20
21 job3:
22   stage: faire_la_cuisine
23   script:
24    - echo "Nettoyer la tête du porc. "
25   tag:
26    - taro
27
28 job4:
29   stage: faire_la_cuisine
30   script:
31    - echo "Piler le taro "
32   tag:
33    - taro
34
35 job5:
36   stage: faire_la_cuisine
37   script:
38    - echo "Faire la sauce. "
39   tag:
40    - taro
41
42 job6:
43   stage: tester
44   script:
45    - echo "Tester s'il n'y a pas trop de sel"
46   tag:
47    - taro
48
49 job7:
50   stage: servir
51   script:
52    - echo "Servir à manger. "
53   tag:
54    - taro
55
```


Practical work on setting up a simple CI/CD pipeline

Configuring the TPtaro project runners

- ▶ Go to the gitlab page and select your project.
- ▶ Open the Tptaro project parameters .click on the CICD option.
- ▶ Expand the runners section and click on the three vertical buttons in front of a new project runner to copy the registration token.
- ▶ Open the gitlab-runner folder and run the gitlab-runner.exe register command. As shown in the last image on the right.

Runners de projet assignés

● #4 (rab14xhg) 🔒

Programme de mise en place d'un plat de taro

taro

✎ || Supprimer le runner

Les runners sont soit :

- **actif** - Disponible pour l'exécution
- **en pause** - Non disponible

Les étiquettes contrôlent les types de runners qui sont capables d'exécuter. En savoir plus

Runners de projet

Ces runners sont assignés à ce projet

Nouvel runner du projet

Jeton d'inscription

⚠ La prise en charge des jetons d'enregistrement est obsolète

GR1348941pQzDRe3yhfVQwRSv2-WD



Afficher les instructions d'installation et d'enregistrement de runner

Réinitialiser le jeton d'inscription

PC > Téléchargements > CC_KEYCE > gitlab-runner

Nom	Modifié le	Type	Taille
.runner_system_id	28/10/2024 00:47	Fichier RUNNER_S...	1 Ko
config	28/10/2024 00:50	Fichier source Toml	1 Ko
gitlab-runner	17/10/2024 16:10	Application	90.817 Ko

```
C:\Windows\System32\cmd.exe
Microsoft Windows [version 10.0.19045.5011]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\TADJEUNIAT\Downloads\CC_KEYCE\gitlab-runner>gitlab-runner register
Runtime platform arch=amd64 os=windows pid=13416 revision=32fe5502 version=17.4.1
Created missing unique system ID system_id=s_2bd805b06d44
Enter the GitLab instance URL (for example, https://gitlab.com/):
http://localhost/
Enter the registration token:
GR1348941pQzDRe3yhfVQwRSv2-WD
Enter a description for the runner:
[DESKTOP-D3MEQ37]: Programme de mise en place d'un plat de taro
Enter tags for the runner (comma-separated):
taro
Enter optional maintenance note for the runner:
ok
WARNING: Support for registration tokens and runner parameters in the 'register' command has been deprecated in GitLab Runner 15.6 and will be replaced with support for authentication tokens. For more information, see https://docs.gitlab.com/ee/ci/runners/new_creation_workflow
Registering runner... succeeded runner=GR1348941pQzDRe3y
Enter an executor: custom, shell, docker-windows, docker+machine, docker-autoscaler, instance, ssh, parallels, virtualbox, x, docker, kubernetes:
shell
Runner registered successfully. Feel free to start it, but if it's running already the config should be automatically reloaded!

Configuration (with the authentication token) was saved in "C:\\Users\\TADJEUNIAT\\Downloads\\CC_KEYCE\\gitlab-runner\\config.toml"

C:\Users\TADJEUNIAT\Downloads\CC_KEYCE\gitlab-runner>
```


Practical work on setting up a CI/CD pipeline simple.

#8200453110

Inviter plus de membres Explorer les forfaits payants

Terminé il y a 9 minutes par FAGUY FAGUY

Rechercher dans la sortie

```
ing with gitlab-runner 17.4.1 (32fe5502)
plat_taro t3_mzi61y, system ID: s_2bd805b06d44
lving secrets
aring the "shell" executor
ing Shell (powershell) executor...
aring environment
ing on DESKTOP-D3MEQ37...
ing source from Git repository
hing changes with git depth set to 20...
itialized existing Git repository in C:/Users/TADJEUNIAT/Downloads/CC_KEYCE/gitlab-runner/builds/t3_mz
/0/grh3422242/taro/.git/
king out 4ae97850 as detached HEAD (ref is main)...
lfs/3.3.0 (GitHub; windows amd64; go 1.19.3; git 77deabdf)
ping Git submodules setup
uting "step_script" stage of the job script
ho "Nettoyer la tête du porc. "
royer la tête du porc.
ning up project directory and file based variables
succeeded
```

Durée: 10 secondes
Terminé: il y a 9 minutes
En file d'attente: 1 seconde
Délai d'attente: 1h (depuis project)
Runner: #43185602 (mziG1y-d)
Programme de mise en place d'un bon plat de taro

Étiquettes: taro

Validation 4ae97850
Erreur syntaxe yaml dans le fichier .gitlab.yml

Pipeline #1515669635 Réussi pour main
faire_la_cuisine

Tâches liées
→ job3
job4
job5

Réussi
00:01:20
il y a une minute

Erreur syntaxe yaml dans le fichier .gitlab.yml
#1515669635 main 4ae97850
plus récent

Profitez au maximum de votre période d'essai en ajoutant plus de membres
Pendant votre période d'essai, invitez autant de membres que vous le souhaitez pour collaborer avec vous sur taro. À la fin de votre essai, vous aurez un maximum de 5 membres avec le forfait Gratuit ; vous pouvez en obtenir davantage en faisant une mise à niveau vers une édition payante.
Inviter plus de membres Explorer les forfaits payants

Tous 2 Terminé Branches Étiquettes View analytics Vider les caches des runners

Filtrer les pipelines

État	Pipeline	Créé par	Étapes
En cours	Erreur syntaxe yaml dans le fichier .gitlab.yml #1515669635 main 4ae97850 plus récent		
En échec	Erreur syntaxe yaml dans le fichier .gitlab.yml #1515666089 main 598292ce		

grh / taro / Pipelines

Profitez au maximum de votre période d'essai en ajoutant plus de membres
Pendant votre période d'essai, invitez autant de membres que vous le souhaitez pour collaborer avec vous sur taro. À la fin de votre essai, vous aurez un maximum de 5 membres avec le forfait Gratuit ; vous pouvez en obtenir davantage en faisant une mise à niveau vers une édition payante.
Inviter plus de membres Explorer les forfaits payants

Tous 2 Terminé Branches Étiquettes View analytics Vider les caches des runners New pipeline

Filtrer les pipelines

État	Pipeline	Créé par	Étapes
En cours	Erreur syntaxe yaml dans le fichier .gitlab.yml #1515669635 main 4ae97850 plus récent		
En échec	Erreur syntaxe yaml dans le fichier .gitlab.yml #1515666089 main 598292ce		

Inviter plus de membres

Explorer les forfaits payants

Erreur syntaxe yaml dans le fichier .gitlab.yml

✓ Réussi **FAGUY FAGUY** a créé le pipeline pour la validation `4ae97850` 📄 il y a 5 minutes, terminé il y a 3 min

Pour `main`

dernier ⌚ 8 jobs ⌚ 1 minute 20 secondes, mis en file d'attente pendant 3 secondes

Pipeline

Jobs 8

Tests 0

aller_a_mar...



job1



job2



faire_la_cuis...



job3



job4



job5



tester



job6



servir



job7



job8



**TP for setting up a CI/CD pipeline
a java-based web application...**

Practical work on setting up a CI/CD pipeline
for a java-based web application and running it
in a docker environment.

END